

Module 2: Spring Boot Web MVC, JPA, Validation, Exception Handling, Lambda, and Streams

Questions

Q1. What is Spring MVC?

Answer: Spring MVC is a web framework within Spring that follows the Model-View-Controller pattern. The Model holds data, the View renders the user interface, and the Controller handles requests and business flow. The core idea is separation of concerns: controllers deal with HTTP, services with logic, and views with presentation. In a banking web portal, the controller receives a login request, the service validates credentials, the model carries account details, and the view renders the dashboard. For REST APIs, the "view" is usually JSON produced by message converters.

Q2. What is the role of `DispatcherServlet` in Spring MVC?

Answer: `DispatcherServlet` is the front controller that receives every HTTP request and acts as the single entry point. Its job is to delegate: it uses handler mappings to find the right controller, handler adapters to invoke it, and view resolvers or message converters to produce the response. It also applies interceptors and centralizes exception handling. In a supply-chain order system, every request to `/orders/*` first hits the `DispatcherServlet`, which then routes it to `OrderController`. Without it, each controller would need its own routing and common concern logic.

Q3. Describe the lifecycle of a Spring MVC request.

Answer: The lifecycle begins when the client sends an HTTP request, which the embedded Tomcat receives and forwards to `DispatcherServlet`. The servlet consults `HandlerMapping` to find the controller method matching the URL and HTTP method. It then invokes the method through a `HandlerAdapter`. The controller may call services, repositories, or external clients. It returns either a `ModelAndView` or a response body. For REST, `HttpMessageConverters` serialize the return object to JSON. For traditional MVC, a `ViewResolver` maps the logical view name to a JSP or Thymeleaf template. Finally, `DispatcherServlet` sends the rendered response back to the client. Interceptors run before and after the controller if configured.

Q4. What is the difference between Spring MVC and Spring Boot Web MVC?

Answer: Spring MVC is the web framework that handles HTTP requests, controllers, view resolution, and interceptors. It requires manual configuration such as registering `DispatcherServlet` in `web.xml` or a Java initializer, configuring view resolvers, and setting up component scanning. Spring Boot Web MVC is Spring MVC with opinionated auto-configuration. When you add `spring-boot-starter-web`, Spring Boot registers `DispatcherServlet`, configures default JSON message converters, starts embedded Tomcat, and enables component scanning automatically. In a traditional banking project using plain Spring

MVC, you write more XML or Java config. In a modern Spring Boot banking microservice, you write the controller and the framework wires the rest.

Q5. What is Tomcat's role in a Spring Boot application?

Answer: Tomcat is the embedded servlet container bundled with Spring Boot Web. It listens for HTTP requests and passes them to `DispatcherServlet`. In production, a banking API runs as a JAR, and the embedded Tomcat starts automatically when the application launches.

Q6. What is the `WebApplicationContext`?

Answer: `WebApplicationContext` is a Spring container specialized for web applications. It holds web-specific beans such as controllers, view resolvers, and interceptors. `DispatcherServlet` creates or inherits this context to resolve and invoke controller beans when requests arrive.

Q7. What is a DTO, and why do you use it?

Answer: A DTO, or Data Transfer Object, is a plain Java object used to carry data between application layers or between the backend and external clients. Its purpose is to decouple the internal domain model from the API contract. In a banking app, instead of exposing a `BankAccount` JPA entity with database IDs, audit fields, and internal flags, you expose an `AccountDto` with only account number, balance, and holder name. DTOs prevent over-fetching of data, protect sensitive fields, reduce payload size, and make versioning easier because you can change the entity schema without changing what clients receive.

Q8. What is the difference between an entity and a DTO?

Answer: An entity maps to a database table and includes persistence annotations. A DTO is a plain object used for API requests and responses. For example, a supply-chain `ShipmentEntity` might have database IDs and audit fields, while a `ShipmentDto` contains only tracking number, origin, and destination.

Q9. What is the role of a controller in Spring Web MVC?

Answer: A controller receives HTTP requests, extracts input data, delegates processing to services, and returns a response or view name. In a banking transfer API, a `TransferController` maps a POST request, validates the DTO, calls `TransferService`, and returns the transaction reference.

Q10. What is the difference between `@Controller` and `@RestController`?

Answer: `@Controller` is a Spring stereotype for web controllers that return view names by default. If you want to return raw data like JSON from a `@Controller` method, you must add `@ResponseBody` to that method. `@RestController` is a convenience annotation that combines `@Controller` and `@ResponseBody` at the class level, so every method automatically writes its return value to the response body. For example, a banking `AccountController` using `@RestController` can return an `AccountDto` object, and Jackson automatically converts it to

JSON without adding `@ResponseBody` on every method.

Q11. What are `@RequestMapping`, `@GetMapping`, `@PostMapping`, `@PutMapping`, `@PatchMapping`, and `@DeleteMapping`?

Answer: These annotations map URLs to controller methods. `@RequestMapping` is generic and can specify any HTTP method. The others are specialized shortcuts: `@GetMapping` for read, `@PostMapping` for create, `@PutMapping` for full update, `@PatchMapping` for partial update, and `@DeleteMapping` for delete.

Q12. What is the difference between `@PathVariable` and `@RequestParam`?

Answer: `@PathVariable` captures values from the URL path, like `/accounts/{id}`. `@RequestParam` reads query parameters or form data, like `/accounts?type=savings`. In a supply-chain API, `/shipments/123` uses `@PathVariable` for the ID, while `/shipments?status=DELIVERED` uses `@RequestParam`.

Q13. What is `@RequestBody` used for?

Answer: `@RequestBody` tells Spring to read the HTTP request body and deserialize it into a Java object. The conversion is done by `HttpMessageConverters`, usually Jackson for JSON. For example, in a banking payment API, a POST body `{ "fromAccount": "12345", "toAccount": "67890", "amount": 1000 }` is automatically mapped to a `PaymentRequestDto` object. The DTO can then be validated and passed to the service. If the JSON is malformed, Spring responds with a 400 Bad Request before the controller method executes.

Q14. What is `@ResponseBody` and how does it work?

Answer: `@ResponseBody` tells Spring to write the controller method's return value directly into the HTTP response body rather than interpreting it as a view name. Spring uses `HttpMessageConverters`, such as Jackson, to serialize the object to JSON or XML. For example, a method returning an `AccountDto` object becomes a JSON response like `{ "accountNumber": "...", "balance": 5000 }`. When you use `@RestController`, every method inherits `@ResponseBody` automatically, which is why REST controllers do not need it on each method.

Q15. What are the PUT, PATCH, and DELETE HTTP methods used for?

Answer: PUT replaces an entire resource, so the client must send the full updated object. PATCH applies a partial update, so only the fields that change are sent. DELETE removes a resource or performs a logical delete. In a banking API, PUT might fully replace a customer profile, PATCH might update only the phone number or address, and DELETE might deactivate an account while keeping the record in the database. Using the right method keeps the API predictable and RESTful.

Q16. When should you use PUT versus PATCH?

Answer: Use PUT when the client is responsible for sending the complete resource, even for fields that do not change. Use PATCH when only a subset of fields needs to change, making the payload smaller and the operation more focused. For a supply-chain product catalog, PUT might replace the entire product object including name, category, price, and stock. PATCH might send only `{ "price": 29.99, "stock": 150 }` to update just those two fields. PATCH reduces network overhead and avoids accidental overwrites of unchanged data.

Q17. What is the role of the service layer in a Spring application?

Answer: The service layer sits between controllers and repositories and owns the business logic of the application. It coordinates multiple repositories, external APIs, and domain rules, and it defines transaction boundaries. A `PaymentService` in a banking app validates transfer limits, calls a fraud-check service, debits the source account, credits the destination account, and logs the transaction. By placing these steps in one service method annotated with `@Transactional`, you ensure that all database changes succeed or fail together. Keeping business logic in the service layer also makes controllers thin, repositories focused on data access, and the application easier to test and maintain.

Q18. How do you keep controllers thin and services fat?

Answer: Controllers should only handle HTTP mapping, input validation, and response building. All business rules, calculations, and data orchestration belong in the service layer. This separation makes the supply-chain order controller easy to read and the `OrderService` easy to unit test.

Q19. What is a repository in Spring Data JPA?

Answer: A repository is an interface that Spring Data JPA implements automatically at runtime. It provides CRUD methods, query derivation, paging, sorting, and custom queries. A banking `TransactionRepository` extends `JpaRepository<Transaction, Long>` gives you `save`, `findById`, and custom finder methods for free.

Q20. What is the difference between `CrudRepository`, `PagingAndSortingRepository`, and `JpaRepository`?

Answer: `CrudRepository` provides basic CRUD operations. `PagingAndSortingRepository` adds pagination and sorting. `JpaRepository` extends both and adds JPA-specific methods like `flush`, `saveAll`, and batch delete. Most Spring Boot projects use `JpaRepository` for its richer feature set.

Q21. How does Spring Data JPA create query methods automatically?

Answer: Spring Data JPA uses a query derivation mechanism. When you declare a method like `findByAccountNumber` or `findByStatusAndCreatedDateAfter` in a repository interface, it parses the method name into a JPQL query. Keywords like `And`, `Or`, `GreaterThan`, `Like`, `OrderBy`, and `IgnoreCase` control the query shape. For complex joins or queries that cannot be expressed by naming convention, you annotate the method with `@Query` and write JPQL or native SQL. For example, in a banking app, `List<Transaction>`

`findByAccountNumberAndTransactionDateBetween(String accountNumber, LocalDate start, LocalDate end)` automatically generates a query that fetches transactions for an account within a date range.

Q22. What is the purpose of pagination in Spring Data JPA?

Answer: Pagination breaks large result sets into smaller pages to save memory and improve response time. You pass a `Pageable` object with page number, size, and sort. A supply-chain shipment search returning millions of rows can return `Page<Shipment>` with only twenty records per request.

Q23. What is the difference between `FetchType.EAGER` and `FetchType.LAZY`?

Answer: `EAGER` loads related entities immediately with the parent. `LAZY` loads them only when accessed. `LAZY` is preferred for performance. In a banking app, eager loading all transactions with every account query can cause the N+1 problem, so lazy loading is usually better.

Q24. What is the N+1 SELECT problem, and how do you avoid it?

Answer: The N+1 problem happens when one query fetches N parent records, and then N additional queries are executed to load related child records. For example, if you fetch 100 supply-chain orders and then lazily access `order.getItems()` for each order, Hibernate performs one query for the orders plus 100 queries for the items. To avoid this, you can use `JOIN FETCH` in a `@Query` annotation to load the parent and children in a single query. Alternatively, you can use entity graphs, batch fetching through `hibernate.default_batch_fetch_size`, or `FetchType.EAGER` carefully where it makes sense. The goal is to load the required data with the minimum number of queries.

Q25. What is input validation in Spring Boot?

Answer: Input validation ensures that request data meets business rules before processing. Spring Boot supports Bean Validation using annotations like `@NotNull`, `@Size`, `@Min`, `@Max`, `@Email`, and `@Pattern`. A banking fund-transfer DTO might require `@NotBlank` account numbers and `@Positive` amount.

Q26. What is the difference between `@Valid` and `@Validated`?

Answer: `@Valid` comes from the Bean Validation standard and is used on request bodies and nested objects. `@Validated` is a Spring extension that supports group validation and can be used at the class or method level. For a supply-chain API, `@Valid` validates a posted order DTO, while `@Validated` can validate method parameters by group.

Q27. How do you create a custom validation annotation in Spring Boot?

Answer: You define an annotation, a `ConstraintValidator` implementation, and link them using `@Constraint`. For example, a custom `@AccountNumber` validator checks that a banking account number matches a specific length and digit pattern. You then place the annotation on a DTO field.

Q28. What is `@ControllerAdvice` and `@RestControllerAdvice`?

Answer: `@ControllerAdvice` is a special annotation that marks a class as a global advisor for multiple controllers. Methods inside it can use `@ExceptionHandler`, `@InitBinder`, and `@ModelAttribute` to apply across controllers. `@RestControllerAdvice` is a convenience variant that combines `@ControllerAdvice` and `@ResponseBody`, so the return values are written directly to the response body as JSON. In a banking app, a single `@RestControllerAdvice` class can catch `MethodArgumentNotValidException` for validation failures, `AccountNotFoundException` for missing accounts, and generic exceptions, returning a consistent error response structure for every endpoint.

Q29. What is `@ExceptionHandler`?

Answer: `@ExceptionHandler` marks a method that handles a specific exception type thrown by controller methods. When that exception occurs, Spring routes control to the handler method instead of letting the exception bubble up as a 500 error. The method typically returns a `ResponseEntity` with an HTTP status and an error body. For example, a method annotated with `@ExceptionHandler(AccountNotFoundException.class)` in a `@RestControllerAdvice` class can return `ResponseEntity.status(HttpStatus.NOT_FOUND).body(errorResponse)`. This keeps error handling centralized and user-friendly.

Q30. How do you handle validation errors in Spring Boot?

Answer: When validation fails, Spring throws `MethodArgumentNotValidException`. A global `@ExceptionHandler` can catch it, read the `BindingResult` to extract field-level errors, and return a structured error response. The response typically contains the HTTP status, a timestamp, the request path, and a list of errors with field names and messages. For example, in a supply-chain API, the client receives `{"errors": [{"field": "quantity", "message": "must be at least 1"}]}` with a 400 status. This avoids raw stack traces and gives clients actionable feedback.

Q31. How do you transform or customize API responses in Spring Web MVC?

Answer: You can customize responses in several ways. Use `ResponseEntity` to set the exact HTTP status, headers, and body. Use DTOs to shape the data sent to the client. Use Jackson annotations like `@JsonProperty` to rename fields, `@JsonIgnore` to exclude fields, and `@JsonFormat` to format dates. Use message converters to support multiple formats like JSON and XML. For example, in a banking API, after creating an account, you return `ResponseEntity.created(location).body(new AccountDto(account))` with a 201 status and a `Location` header pointing to the new resource.

Q32. What is `ResponseEntity`?

Answer: `ResponseEntity` is a Spring class that represents the complete HTTP response, including status code, headers, and body. It lets a controller decide exactly what the client receives. For example,

`ResponseEntity.created(URI.create("/orders/123")).body(orderDto)` returns 201 Created, adds a Location header pointing to the new resource, and includes the created order data in the body. For errors, you can return `ResponseEntity.status(HttpStatus.BAD_REQUEST).body(errorResponse)`. This is much more flexible than simply returning a DTO.

Q33. How do you use Jackson annotations to shape JSON responses?

Answer: Jackson annotations let you control serialization without changing the service logic. Use `@JsonProperty("customer_name")` to change the JSON key name. Use `@JsonIgnore` to prevent a field from appearing in the response, such as an internal database ID. Use `@JsonFormat(pattern = "yyyy-MM-dd", shape = JsonFormat.Shape.STRING)` to format dates. Use `@JsonInclude(JsonInclude.Include.NON_NULL)` to omit null fields and reduce payload size. In a banking API, you might hide the internal `accountId` and expose only `accountNumber`, while formatting transaction timestamps to a readable string.

Q34. What is a lambda expression in Java 8?

Answer: A lambda expression is a concise way to represent an anonymous function that implements a functional interface. It has parameters, an arrow, and a body. For example, `list.forEach(item -> System.out.println(item))` replaces a verbose anonymous inner class.

Q35. What is a functional interface?

Answer: A functional interface has exactly one abstract method. Java 8 provides built-in ones like `Predicate`, `Function`, `Consumer`, and `Supplier`. The `@FunctionalInterface` annotation marks them and prevents adding extra abstract methods. A supply-chain `Predicate<Order>` can test if an order is urgent.

Q36. What are intermediate and terminal operations in the Stream API?

Answer: Intermediate operations transform a stream into another stream and are lazy, meaning they do not execute until a terminal operation is called. Examples include `filter`, `map`, `flatMap`, `sorted`, `distinct`, and `limit`. Terminal operations consume the stream and produce a result, such as `collect`, `forEach`, `reduce`, `count`, and `findFirst`. For example, in a banking report, `transactions.stream().filter(t -> t.getType().equals("DEBIT")).map(Transaction::getAmount).collect(Collectors.toList())` does nothing until `collect` runs. Then the entire pipeline executes efficiently in one pass.

Q37. What is the difference between map and flatMap in streams?

Answer: `map` transforms each element into exactly one new element, preserving the one-to-one relationship. `flatMap` transforms each element into a stream of zero or more elements, and then flattens all those nested streams into a single stream. For example, if you have `List<Order>` and each `Order` contains `List<OrderItem>`, `orders.stream().map(Order::getItems)`

gives you a stream of lists. `orders.stream().flatMap(order -> order.getItems().stream())` gives you a single stream of all `OrderItem` objects. This is useful for nested data in supply-chain order processing.

Q38. How do you filter and collect data using streams?

Answer: Use `filter` with a predicate to keep matching elements, then `collect` with a collector like `Collectors.toList()`. For example, `transactions.stream().filter(t -> t.getAmount() > 10000).collect(Collectors.toList())` finds high-value banking transactions.

Q39. What is the purpose of reduce in streams?

Answer: `reduce` combines stream elements into a single value using an accumulator. For example, `amounts.stream().reduce(0, Integer::sum)` adds all transaction amounts. It is useful for aggregations like totals, averages, and maximums.

Q40. How do you group or partition data with streams?

Answer: Use `Collectors.groupingBy` to group elements by a key, such as grouping transactions by status. Use `Collectors.partitioningBy` to split into two groups based on a predicate. A supply-chain app might group shipments by destination country for a summary report.

Q41. What is Optional, and why is it useful?

Answer: `Optional` is a wrapper introduced in Java 8 that represents a value that may or may not be present. It forces the caller to explicitly handle the absence of a value instead of risking a `NullPointerException`. For example, a repository method can return `Optional<Account>` instead of an `Account` that might be null. The service can then call `.orElseThrow(() -> new AccountNotFoundException("Account not found"))` or `.ifPresent(account -> process(account))`. This makes the code safer and more readable, especially in banking lookups where missing data is a real possibility.

Q42. What are method references, and when do you use them?

Answer: Method references are shorthand for lambdas that simply call an existing method. For example, `names.stream().map(String::toUpperCase)` is shorter than `names.stream().map(s -> s.toUpperCase())`. Use them when a lambda only calls one method without extra logic.

Q43. What is lazy evaluation in streams?

Answer: Lazy evaluation means intermediate operations are not executed until a terminal operation is called. This allows the stream to optimize by fusing operations and avoiding unnecessary work. For a large supply-chain dataset, filtering before mapping reduces the number of transformations.

Q44. What are peek, limit, and skip used for in streams?

Answer: `peek` inspects elements for debugging without changing them. `limit` truncates the

stream to a maximum size. `skip` discards the first N elements. In a banking transaction history, `skip(10).limit(20)` implements simple pagination.

Q45. How do you find duplicates in a list using streams?

Answer: You can use `Collectors.groupingBy` and then filter groups with more than one element, or use a `Set` to track seen items. For example, `list.stream().filter(i -> Collections.frequency(list, i) > 1).collect(Collectors.toSet())` finds duplicate account numbers, though it is not the most efficient for huge lists.

Q46. What is a HandlerInterceptor, and when do you use it?

Answer: A `HandlerInterceptor` is a Spring MVC mechanism that intercepts HTTP requests within the Spring lifecycle. It has three methods: `preHandle` runs before the controller and can block the request by returning false; `postHandle` runs after the controller but before the view is rendered; `afterCompletion` runs after the entire request is complete and is useful for cleanup or final logging. In a banking API, you can use an interceptor to log every incoming request, add a correlation ID to the response, or check a custom security header before the controller executes. It is registered globally through `WebMvcConfigurer.addInterceptors`.

Q47. How do you serve static resources in Spring Boot Web MVC?

Answer: Spring Boot serves static files from `/static`, `/public`, `/resources`, or `/META-INF/resources` by default. You can also register custom resource handlers with `WebMvcConfigurer.addResourceHandlers`. In a supply-chain portal, PDF invoices might be served from the `/static/invoices` folder.

Q48. What is content negotiation in Spring MVC?

Answer: Content negotiation lets a controller produce different response formats, such as JSON or XML, based on the client's `Accept` header. Spring uses `ContentNegotiatingViewResolver` and message converters. A banking partner requesting XML can receive the same data as another partner requesting JSON.

Q49. What is the difference between a filter and an interceptor?

Answer: A servlet filter operates at the servlet container level, before the request reaches `DispatcherServlet`. It is part of the Java Servlet API and can wrap any request or response, making it suitable for low-level tasks like request logging, CORS handling, compression, or security headers. An interceptor operates inside the Spring MVC lifecycle, after `DispatcherServlet` has received the request. It has access to the controller handler, the model, and the view, making it suitable for Spring-aware concerns like adding model attributes, authentication checks, or post-processing responses. In a banking app, a filter might add security headers to every response, while an interceptor might check a per-endpoint permission annotation.

Q50. How do you write a unit test for a Spring MVC controller?

Answer: I annotate the test class with `@WebMvcTest(TransferController.class)` so Spring loads only the web layer components needed for that controller. I inject `MockMvc` to

simulate HTTP requests without starting a real server. I use `@MockBean` to replace the service layer with a Mockito mock. In the test, I configure the mock with `when(transferService.transfer(...)).thenReturn(transactionReference)`, then call `mockMvc.perform(post("/transfers").contentType(MediaType.APPLICATION_JSON).content(jsonBody))`. I verify the HTTP status, response body, and sometimes the interaction with the mock using `verify(transferService, times(1)).transfer(...)`. This tests the controller in isolation and runs quickly because no database or external service is started.

Q51. What is `@WebMvcTest` used for?

Answer: `@WebMvcTest` is a test slice that loads only Spring MVC components for a specific controller, without starting the full application context or database. It makes controller tests fast. In a supply-chain project, you test `OrderController` without loading `OrderRepository`.

Scenario-Based Questions

S1. A banking portal receives an HTTP POST to `/accounts/{id}/transfer`. Walk me through the Spring MVC flow.

Answer: The flow starts when the client sends the POST request with JSON body and account ID. Tomcat accepts the connection and forwards the request to `DispatcherServlet`. `DispatcherServlet` consults handler mappings to find `AccountController.transferFunds`. It invokes the method through a handler adapter. The `@PathVariable` extracts the account ID and `@RequestBody` deserializes the JSON into a `TransferRequestDto`. The controller calls `TransferService`, which validates balances, calls the fraud engine, and processes the transfer inside a transaction. The service returns a transaction reference. The controller builds a `ResponseEntity` with 201 Created and a Location header. Jackson converts the DTO to JSON, and `DispatcherServlet` sends the response back to the client.

S2. A supply-chain client wants to update only the delivery date of a shipment. Which HTTP method and Spring annotation do you use?

Answer: Use PATCH with `@PatchMapping("/shipments/{shipmentId}")`. PATCH is designed for partial updates, meaning the client sends only the fields that need to change. The controller maps the shipment ID with `@PathVariable` and the request body with `@RequestBody`. The DTO contains only `deliveryDate` and perhaps a version field for optimistic locking. The service loads the existing shipment, applies the change, and saves. Because PUT would require the client to send the entire shipment object, PATCH is the cleaner choice here. The response typically returns 200 OK with the updated shipment DTO.

S3. A bank needs to expose account details but must hide the internal database ID and the customer's full address. How do you handle this?

Answer: The safest approach is to introduce a dedicated `AccountDto` that contains only the fields the API contract allows, such as account number, account type, available balance, and a masked phone number. In the service layer, I map the `Account` entity to the DTO using a mapper like `MapStruct` or a manual mapping method. The controller returns the DTO, never the entity. This prevents internal fields like the database ID, full address, and audit metadata from leaking to clients. It also decouples the database schema from the API contract, so future schema changes do not break external consumers.

S4. A supply-chain API accepts a huge CSV upload of orders. How do you validate each row before saving?

Answer: I process the CSV in a streaming manner to avoid loading everything into memory. For each row, I map it to an `OrderImportDto` and run Bean Validation. Fields like product code use `@NotBlank`, quantity uses `@Positive`, and delivery date uses `@Future` or a custom date validator. I use a `Validator` instance directly or validate programmatically. I collect all failures in a list with row number, field, and message. Valid rows are grouped and processed in batches using `saveAll`. At the end, the API returns a summary such as total rows, successful imports, failed rows, and the detailed error list. This ensures no partial or invalid data enters the database.

S5. A banking API returns 500 Internal Server Error for every invalid request. How do you improve the client experience?

Answer: The problem is that unhandled exceptions default to 500, which hides the real cause. I add a `@RestControllerAdvice` class with specific `@ExceptionHandler` methods. For `MethodArgumentNotValidException`, I return 400 Bad Request with a list of field errors and messages. For `AccountNotFoundException`, I return 404 Not Found with the account identifier. For business rule violations, I return 409 Conflict or 422 Unprocessable Entity. For unexpected errors, I return 500 but log the stack trace internally and expose only a generic reference ID. The response body always follows the same structure: status, timestamp, path, message, and optional details. This gives clients clear, actionable feedback instead of a generic server error.

S6. A supply-chain search endpoint returns 100,000 shipment records, and the UI freezes. How do you fix it?

Answer: Implement pagination with `Pageable` in the repository method. Return `Page<ShipmentDto>` with only the requested page, size, and sort. Add default limits and validate that clients cannot request an excessively large page size. This reduces memory usage and improves response time.

S7. A banking fund-transfer DTO has many fields, and validation rules differ for retail versus corporate customers. How do you manage this?

Answer: Use validation groups. Define interfaces like `RetailValidation` and `CorporateValidation`. Annotate DTO fields with `@NotNull(groups = CorporateValidation.class)` and use

@Validated(CorporateValidation.class) on the controller method. This applies different rules based on the customer type without duplicating DTOs.

S8. A supply-chain order response needs to show total amount, currency, and item count in a custom shape that does not match the entity. How do you build it?

Answer: I create a dedicated OrderSummaryDto with fields like orderNumber, totalAmount, currency, and itemCount. In the service layer, I fetch the Order entity along with its line items, either eagerly or via a fetch join. I then compute the total by summing `unitPrice * quantity` across items, determine the currency from the order header, and count the number of line items. I map these computed values into the DTO. The controller returns OrderSummaryDto instead of the raw entity. This gives the client a clean, purpose-built shape without exposing internal persistence details or forcing the client to do its own calculations.

S9. A banking transaction report must group transactions by type and show the sum for each type. How do you implement this with streams?

Answer: I use `transactions.stream().collect(Collectors.groupingBy(Transaction::getType, Collectors.summarizingDouble(Transaction::getAmount)))`. This returns a `Map<String, DoubleSummaryStatistics>` where the key is the transaction type and the value contains sum, count, average, min, and max. From there, I iterate over the map entries and build a list of TransactionSummaryDto objects with fields for type, totalAmount, transactionCount, and averageAmount. The service returns this list, and the controller serializes it to JSON. This approach keeps the grouping and aggregation concise without writing explicit loops.

S10. A supply-chain API needs to return the top five highest-value orders placed in the last 30 days. Write the stream pipeline.

Answer: `orders.stream().filter(o -> o.getOrderDate().isAfter(LocalDate.now().minusDays(30))).sorted(Comparator.comparing(Order::getTotalAmount).reversed()).limit(5).collect(Collectors.toList())`. This filters by date, sorts descending by amount, limits to five, and collects the result.

S11. A developer is calling repository.findAll() and loading millions of banking transactions into memory. What is the risk and the fix?

Answer: The risk is an OutOfMemoryError and poor performance. The fix is to use pagination with Pageable, or stream results with Stream<Transaction> and process them in batches. For reports, consider database-level aggregation or batch processing instead of loading all rows at once.

S12. A supply-chain order has a list of line items, and you need a single list of all product IDs across orders. How do you use flatMap?

Answer: I write `orders.stream().flatMap(order -> order.getItems().stream()).map(OrderItem::getProductId).distinct().collect(Collectors.toList())`. First, `flatMap` takes each `Order`, gets its `List<OrderItem>`, and turns that nested list into a single continuous stream of `OrderItem` objects. Then `map` extracts the product ID from each item. `distinct` removes duplicates so the same product appearing in multiple orders appears only once. Finally, `collect` builds the result list. This is a classic pattern for flattening one-to-many relationships in domain models.

S13. A banking client sends a JSON payload where some dates must be formatted as yyyy-MM-dd HH:mm. How do you enforce this?

Answer: Use `@JsonFormat(pattern = "yyyy-MM-dd HH:mm", shape = JsonFormat.Shape.STRING)` on the DTO date field. Alternatively, register a custom `Jackson2ObjectMapperBuilderCustomizer` bean for global date formatting. This ensures every date in the response follows the required pattern.

S14. A supply-chain shipment search can filter by status, destination, or date range. How do you build a flexible query in Spring Data JPA?

Answer: If the filter combinations are fixed and small, I use derived query methods like `findByStatusAndDestinationAndShipmentDateBetween`. For dynamic filters where any combination can be null or present, I use the JPA Specification API. I make the repository extend `JpaSpecificationExecutor<Shipment>`, then build a `Specification<Shipment>` in the service by chaining predicates. Each predicate is only added if the corresponding request parameter is present. For example, status gets an equal predicate, destination gets a like predicate, and the date range gets a between predicate. The repository then calls `findAll(spec, pageable)` to return filtered, paginated results. This keeps the query construction flexible and avoids writing many overloaded repository methods.

S15. A banking API must reject requests where the transfer amount exceeds the customer's daily limit. Where do you enforce this rule?

Answer: Enforce it in the service layer, not the controller or DTO validation, because it requires business data such as the day's previous transfers. The service queries the transaction history, calculates the running total, and throws a domain exception if the new transfer would exceed the limit.

S16. A Spring Boot application starts but all controller endpoints return 404. What do you check?

Answer: First, I check that the controller class is annotated with `@RestController` or `@Controller` and that the URL mappings are correct. Next, I verify that the controller package is under the component scan path defined by the main class with `@SpringBootApplication`. If the controller is in a different package, I add an explicit `@ComponentScan`. I also check that the application type is not set to non-web via `spring.main.web-application-type=none`. If

there is a custom `WebMvcConfigurationSupport` bean, it disables Spring Boot's auto-configuration and may remove default static resource handling or message converters. Finally, I inspect the actuator `/beans` endpoint or log the loaded controllers to confirm they were registered.

S17. A supply-chain partner receives XML instead of JSON from the same endpoint. How does Spring handle this?

Answer: Spring uses content negotiation to decide the response format based on the client's `Accept` header and the available message converters. If the client sends `Accept: application/xml`, Spring looks for a converter that can produce XML. Adding `jackson-dataformat-xml` to the classpath registers an XML-capable message converter. The controller method still returns the same DTO, but the selected converter serializes it to XML instead of JSON. The endpoint URL does not need to change. This works as long as the DTO has a public no-arg constructor and public getters, which Jackson requires for both JSON and XML serialization.

S18. A developer wants to log every incoming banking request and the time it took. Which Spring MVC component is best?

Answer: A `HandlerInterceptor` is the best choice because it runs inside the Spring MVC lifecycle and has access to the controller handler and request attributes. I create a class implementing `HandlerInterceptor` and override `preHandle` to capture the start time and store it in a request attribute. In `afterCompletion`, I retrieve the start time, calculate the duration, and log the HTTP method, URI, status code, and elapsed milliseconds. I register the interceptor globally by implementing `WebMvcConfigurer` and overriding `addInterceptors`. This gives consistent request logging across all controllers without polluting each method with timing code. A servlet filter is an alternative, but it has less context about the Spring handler.

S19. A custom validation rule checks whether a banking account number is active. How do you implement it?

Answer: I implement it as a custom Bean Validation constraint. First, I create an annotation `@ActiveAccount` annotated with `@Constraint(validatedBy = ActiveAccountValidator.class)`. Then I write the `ActiveAccountValidator` class implementing `ConstraintValidator<ActiveAccount, String>`. Inside `isValid`, I inject `AccountRepository` and check whether the account number exists and its status equals `ACTIVE`. I place `@ActiveAccount` on the DTO field, such as `private String fromAccountNumber`. When the controller parameter is annotated with `@Valid`, Spring runs all validators including this custom one before calling the service. If validation fails, Spring throws `MethodArgumentNotValidException`, which the global exception handler converts to a 400 response.

S20. A supply-chain service must process a list of delayed orders and send a notification for each. Write a concise stream-based approach.

Answer: `orders.stream().filter(o -> "DELAYED".equals(o.getStatus())).forEach(notificationService::send`

`DelayAlert()`). This filters delayed orders and invokes the notification service for each. If the notification is slow, replace `forEach` with `parallelStream` cautiously or hand it off to an async task executor.

S21. A banking report endpoint currently returns entities directly. The audit team asks you to hide certain fields. What is the safest approach?

Answer: Introduce DTOs and map only the fields permitted by the audit policy. Use `MapStruct` or a manual mapper to convert from entity to DTO. Remove any `@JsonIgnore` band-aids on entities, because entities should not be shaped by API concerns. Return the DTO from the controller.

S22. A PATCH request to update a banking customer profile fails because required fields are missing. Is this expected?

Answer: Yes, this is expected if validation annotations require fields that are not being sent. For PATCH, the DTO should only contain updatable fields, and validation groups should mark those fields as optional for the patch group. Alternatively, use a separate `CustomerPatchDto` with relaxed validation and apply `@Null` or optional checks explicitly.

S23. A supply-chain endpoint returns `LazyInitializationException` after moving to a REST controller. Why?

Answer: The exception happens because a lazily loaded collection, such as `order.getItems()`, is accessed after the Hibernate session has closed. In a REST controller, Jackson serializes the returned entity to JSON. If Jackson tries to read a lazy collection during serialization, but the transaction and persistence context are already closed, Hibernate cannot fetch the data and throws `LazyInitializationException`. The proper fix is to load all needed data inside the service's transaction before returning it. Options include using `JOIN FETCH` in the repository query, defining an entity graph with `@EntityGraph`, using a DTO projection with `@Query`, or mapping the entity to a DTO within the transaction so Jackson only sees already-loaded data. Returning entities directly from REST controllers is generally discouraged for this reason.

S24. A banking application must ensure no duplicate account numbers are submitted in a batch import. How can streams help?

Answer: Convert the list to a stream and collect into a `Set` to detect duplicates, or use `Collectors.groupingBy` and filter groups with count greater than one. For large batches, combine stream checking with a database unique constraint as the final guard against duplicates.

S25. A Spring Boot test for a controller fails because the service is not a bean. What annotation fixes this?

Answer: Add `@MockBean` on the service field inside the `@WebMvcTest` class. Spring replaces the real service with a Mockito mock in the test context. Then configure stubbing with `when(service.method(...)).thenReturn(...)` to control the controller's behavior during the test.